

Kubernetes for ML Inference & GPU Workloads

Week 3 Lab Guide

What students will learn

- How Kubernetes readiness/liveness probes keep traffic away from a pod that isn't ready yet.
- How GPU scheduling works: node labels, taints/tolerations, and the nvidia.com/gpu extended resource.
- The difference between ReadWriteOnce and ReadWriteMany storage, and a classic PyTorch container gotcha.
- How ResourceQuota and LimitRange govern a shared, multi-tenant cluster.
- What the NVIDIA GPU Operator automates on a real GPU cluster.

Before we start

This guide assumes Docker Desktop is installed with the WSL2 backend and Kubernetes enabled (Settings → Kubernetes → Enable Kubernetes). Demos 1, 3, and 4 run on this cluster alone. Demo 2 (the GPU pod) needs a second, GPU-aware cluster — see the note in that section.

Verify your environment first:

```
docker --version
kubectl config current-context    # should print: docker-desktop
kubectl get nodes                 # should show a Ready control-plane node
```

Demo 1 — Inference Service

Folder: 01_inference_service | Files: app.py, Dockerfile, tiny-inference-deployment.yaml

What this teaches

A minimal FastAPI + PyTorch model server. The key idea: the `/health` endpoint returns HTTP 503 until the model has finished loading inside the app's startup (lifespan) handler, and only returns 200 once it's actually ready to serve. This is exactly why Kubernetes readiness and liveness probes exist — a pod should not receive traffic the moment its container starts; it should receive traffic once the application says it's ready.

How to run it

Step 1 — build the image and prove it works standalone:

```
cd 01_inference_service
docker build -t tiny-inference:local .
docker run -d --name test -p 8000:8000 tiny-inference:local
curl http://localhost:8000/health
docker rm -f test
```

Step 2 — deploy it to Kubernetes (2 replicas, with readiness/liveness probes wired to `/health`):

```
kubectl apply -f tiny-inference-deployment.yaml
kubectl rollout status deployment/tiny-inference
kubectl get pods -l app=tiny-inference
```

Step 3 — call it through the cluster Service, not the container directly:

```
kubectl port-forward svc/tiny-inference-svc 8001:8000
# in a second terminal:
curl http://localhost:8001/predict -X POST -H "Content-Type: application/json" \
  -d '{"features":[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]}'
```

Expected output

```
{"status":"ok","device":"cpu"}
{"probs":[0.165,0.302,0.533],"latency_ms":2.26}
```

Demo 2 — GPU Pod

Folder: 02_gpu_pod | File: gpu-smoke.yaml

What this teaches

The minimum YAML needed to get a pod scheduled onto a GPU node: a nodeSelector that pins the pod to nodes labelled as GPU workers, a toleration for the taint GPU nodes usually carry (so only GPU-requesting workloads land there), and resources.limits["nvidia.com/gpu"]: 1 — the extended resource that the NVIDIA device plugin advertises to the scheduler.

Important: this demo needs a GPU-aware cluster

Docker Desktop's built-in Kubernetes has no NVIDIA device plugin, so it cannot schedule nvidia.com/gpu at all — the pod will sit Pending forever there. This demo needs minikube running with GPU support instead (see the setup scripts in the session materials).

How to run it

Label a node so the pod's nodeSelector matches it, then apply the manifest:

```
kubectl label node minikube accelerator=nvidia --overwrite
kubectl apply -f gpu-smoke.yaml
kubectl logs gpu-smoke
```

Expected output

A real nvidia-smi table, printed from inside a pod that Kubernetes scheduled using the GPU as an extended resource:

```
+-----+
| NVIDIA-SMI 590.48.01      Driver Version: 591.59      CUDA Version: 13.1 |
|-----+-----+-----+
|   0   NVIDIA GeForce RTX 3050 ...    00000000:01:00.0 Off |          N/A |
+-----+
```

On a laptop, minikube's own node is itself a nested Docker container (Windows → WSL2 → Docker Desktop's VM → minikube node → pod). At that depth, containerd's automatic NVIDIA library injection does not fire for Kubernetes-scheduled pods, even though GPU scheduling itself works correctly (kubectl describe node genuinely shows nvidia.com/gpu: 1). The fix needed here was to manually mount the node's own driver libraries into the pod. On a real GPU cluster with the NVIDIA GPU Operator (see Demo 5), none of this manual plumbing is necessary — it's a byproduct of nested virtualization specifically, and a good illustration of how many layers GPU scheduling depends on.

questions :

- What happens if you remove the nodeSelector — where does the scheduler try to place the pod?
- What happens if you request nvidia.com/gpu: 2 on a node that only has 1 GPU? (Try it — describe the pod's events.)
- Why do GPU nodes typically carry a taint at all? What problem does that prevent?

